

生物多样性曲线优化问题实验报告

1. 项目框架概览

本项目围绕 EarthBench / CONOPLib 生物多样性曲线优化问题展开。问题的核心是：给所有物种的首现事件 FAD 和末现事件 LAD 排序，使该排序与地层剖面中的化石观测数据尽可能一致。解是一个长度为 `dims` 的排列，元素为 `0..dims-1`，其中偶数表示某个 taxon 的 FAD，奇数表示对应 LAD。评价函数由 CONOPLib 动态库计算 mismatch penalty，代码中取负数作为 fitness，因此 `y_best` 越接近 0 越好。

项目主要结构如下：

- `configs/`：Hydra 配置，包含任务配置和算法配置。
- `scripts/`：运行脚本，例如 `scripts/run_earth_124.sh`。
- `benchmarks/`：问题定义与黑盒评估，`benchmarks/CONOPLib_*/call.py` 调用 `CONOPLib.so`。
- `algorithms/`：优化算法实现，包括 EA、MAP-Elites、BO、RL 相关算法等。
- `Environments/`：强化学习算子训练/推理使用的环境。
- `main.py`：主训练入口，负责创建任务、实例化算法、Ray 并行评估、记录日志和保存结果。

当前实验主要沿 `scripts/run_earth_124.sh -> main.py -> algorithms/_map_elites.py -> algorithms/_ea_operator.py -> benchmarks/CONOPLib_124/call.py` 这条路径进行。每轮中，算法先通过 `ask()` 生成候选解，再由 `main.py` 中的 Ray worker 调用 benchmark 评估，最后通过 `tell()` 把 fitness 回传给算法更新 archive。

评估过程中，`EarthBenchmark.__call__()` 会先对候选解执行 `repair()`，修复 FAD/LAD 和 coexistence 约束，然后调用 `CONOPLib.so` 计算 penalty。为了保证保存的最优解合法，我们已将主流程改为保存 benchmark 返回的修复后解。

2. 交叉与变异算子

项目中的排列算子主要位于 `algorithms/_ea_operator.py`。

2.1 Mutation 算子

`swap_mutation`：随机选择两个位置交换。扰动较小，适合局部微调，但探索能力有限。

`insert_mutation`：随机选择一个元素，将其插入到另一个位置。该操作适合事件排序问题，因为它直接改变某个事件在整体年代序列中的位置，同时较好保留其余事件的相对顺序。

`reversal_mutation`：随机选择一段区间并反转。扰动较强，适合跳出局部最优，但可能较大幅度破坏已有结构。

`shuffle_mutation`：随机选择若干元素并打乱。探索性较强，适合增加多样性，但也可能引入较多无效扰动。

`shift_mutation`：在局部邻域内移动或交换某个元素。扰动范围较小，偏向局部搜索。

2.2 Crossover 算子

`order_crossover`：保留父代 1 的一个片段，其余位置按父代 2 中的相对顺序补齐。该方法强调保留相对顺序，适合排列优化。

`pmx_crossover`：部分映射交叉，保留父代 1 的片段，并用映射关系处理重复冲突。该方法能同时保留部分位置结构和片段结构。

`cycle_crossover`：根据两个父代之间的位置-元素循环进行继承，交替使用父代片段。该方法能保留父代中的位置关系，但有时变化幅度偏保守。

在 `earth_124` 实验中，baseline 使用 `pmx_crossover + insert_mutation`。从问题性质看，`insert` 对事件排序较自然，因此它本身就是很强的基础变异算子。

3. 任务一：不同演化算子的比较设计

作业任务一要求比较不同演化算子对优化效果的影响。由于本问题的解是一个长度为 `dims` 的排列，算子的核心作用是改变事件的相对顺序。我们计划比较框架中已有的 5 个 mutation 算子和 3 个 crossover 算子，并在其他设置尽量一致的情况下观察 `y_best` 收敛曲线、最终最优值和评价次数。

3.1 Mutation 算子对比

`swap mutation` 随机选择两个位置并交换。它的优点是扰动很小、实现简单，适合在已有较好解附近做局部微调；缺点是单次移动能力有限，如果某个事件需要跨越较长距离，可能需要很多次 swap 才能完成。

`insert mutation` 随机选择一个事件，并将其插入到另一个位置。它适合排序问题，因为它能直接改变单个事件在全局序列中的位置，同时基本保留其他事件的相对顺序。对于 FAD/LAD 事件排序问题，某个物种事件位置偏差较大时，insert 往往比 swap 更有效。缺点是如果需要同时调整一组事件，单个 insert 仍然偏局部。

`reversal mutation` 随机选择一段连续区间并反转。它的扰动范围比 swap 和 insert 更大，可能帮助跳出局部最优；但在本问题中，连续区间反转容易破坏已有的 FAD/LAD 顺序和局部年代结构，修复后产生的有效变化可能较难控制。

`shuffle mutation` 随机选择若干个事件并打乱。它的探索性强，能快速制造多样性；缺点是方向性较弱，容易破坏已经形成的高质量排序结构。该算子更适合前期探索，不一定适合后期精修。

`shift mutation` 在局部邻域内移动或交换某个事件。它的优点是扰动温和，适合精细调整；缺点是探索范围较小，在搜索陷入平台时可能难以产生足够大的结构变化。

整体来看，`insert` 更符合本问题“事件在时间序列中移动”的结构特征，因此预期会是较强 baseline；`swap` 和 `shift` 更偏局部微调；`reversal` 和 `shuffle` 更偏探索，但可能带来较多破坏性扰动。

3.2 Crossover 算子对比

`order crossover` 保留父代 1 的一个连续片段，然后按照父代 2 中的相对顺序补齐剩余事件。它强调继承父代的相对顺序信息，适合排列问题。优点是能较好保留顺序结构；缺点是对具体位置结构的保留相对弱。

`pmx crossover` 保留父代 1 的部分片段，并通过映射关系处理重复元素。它同时保留一段位置结构和父代之间的对应关系。优点是对排列合法性友好，且能继承较明确的片段结构；缺点是映射过程可能引入不直观的位置变化。

`cycle crossover` 根据两个父代之间的元素-位置循环来构造子代。它能较强地保留父代中的绝对位置关系。优点是继承结构清晰；缺点是有时变化幅度偏保守，子代可能与某个父代较为接近，探索能力不足。

对本问题而言，crossover 的有效性取决于父代中是否存在可复用的局部排序片段。如果父代 archive 已经包含若干较高质量解，pmx 和 order 都可能继承有用片段；如果父代差异较大，过强的位置继承也可能造成冲突，最终仍需要依赖 benchmark 的 repair 过程修复约束。

3.3 Mutation 算子对比实验结果

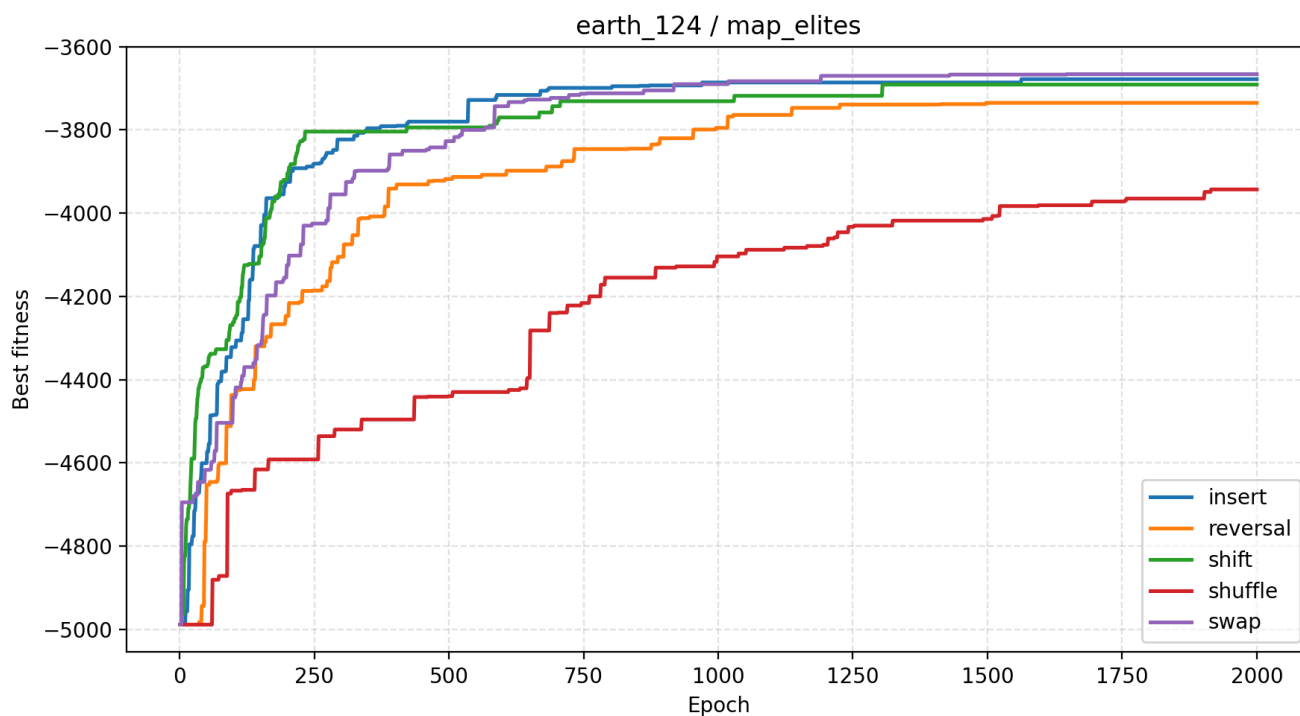
我们首先完成了 5 个 mutation 算子的对比实验。该组实验统一使用：

- `algorithm = map_elites`
- `selection_type = tournament`
- `tournament_size = 3`
- `crossover_type = pmx`
- `correlation_threshold = 0.68`
- `archive_size = 20`
- `pop_size = 20`
- `init_sampler_type = init_v3`
- 不启用 adaptive mutation、动态 correlation threshold、局部搜索、LNS、threshold accepting 和 diversity bonus。

实验结果保存在：

`results/earth_124/map_elites1/`

对比曲线如下：



各 mutation 算子的最终结果如下：

mutation	total evaluations	archive size	final y_best	排名
swap	39964	20	-3666	1
insert	39964	20	-3678	2
shift	39964	15	-3691	3
reversal	39964	20	-3735	4
shuffle	39964	20	-3943	5

从最终结果看，swap 在本次实验中表现最好，最终达到 `y_best = -3666`，超过原先使用 insert 的 baseline `-3678`。insert 仍然表现稳定，排名第二。shift 的最终值为 `-3691`，略弱于 insert，且最终 archive size 只有 15，说明其产生的新解可能更容易与已有 archive 个体相似，导致多样性填充不足。reversal 和 shuffle 明显较弱，尤其 shuffle 最终只有 `-3943`。

从收敛过程看，不同 mutation 的特点也比较明显：

epoch	swap	insert	shift	reversal	shuffle
100	-4443	-4322	-4262	-4437	-4667
250	-4025	-3881	-3804	-4186	-4592
500	-3827	-3780	-3794	-3918	-4440
750	-3713	-3699	-3731	-3846	-4216
1000	-3690	-3686	-3731	-3795	-4104
1500	-3667	-3686	-3691	-3735	-4014
2000	-3666	-3678	-3691	-3735	-3943

insert 和 shift 在前期收敛较快，能够较早把解推进到 `-3800` 附近。swap 前期略慢，但中后期持续改进，在 `epoch=1191` 左右超过 `-3678`，并最终达到本组实验最好结果。这个现象说明，对 earth_124 而言，较小、稳定的局部扰动反而可能更适合后期精修。

reversal 和 shuffle 的表现较差，主要原因可能是扰动过强。reversal 会反转连续区间，容易破坏已有的 FAD/LAD 局部顺序；shuffle 会随机打乱多个事件，虽然增加了探索性，但缺乏方向性，修复后仍难以稳定继承父代中的有效排序结构。因此在当前设置下，强探索型 mutation 并没有带来更好的最终结果。

总体来看，本组实验表明：在 earth_124 的 MAP-Elites 框架下，小扰动算子更有优势，其中 swap 的后期精修能力最好，insert 是稳定且较强的 baseline，shuffle/reversal 这类强扰动算子不适合作为单一 mutation 长时间使用。

3.4 Crossover 算子对比实验结果

随后我们完成了 3 个 crossover 算子的对比实验。该组实验统一使用：

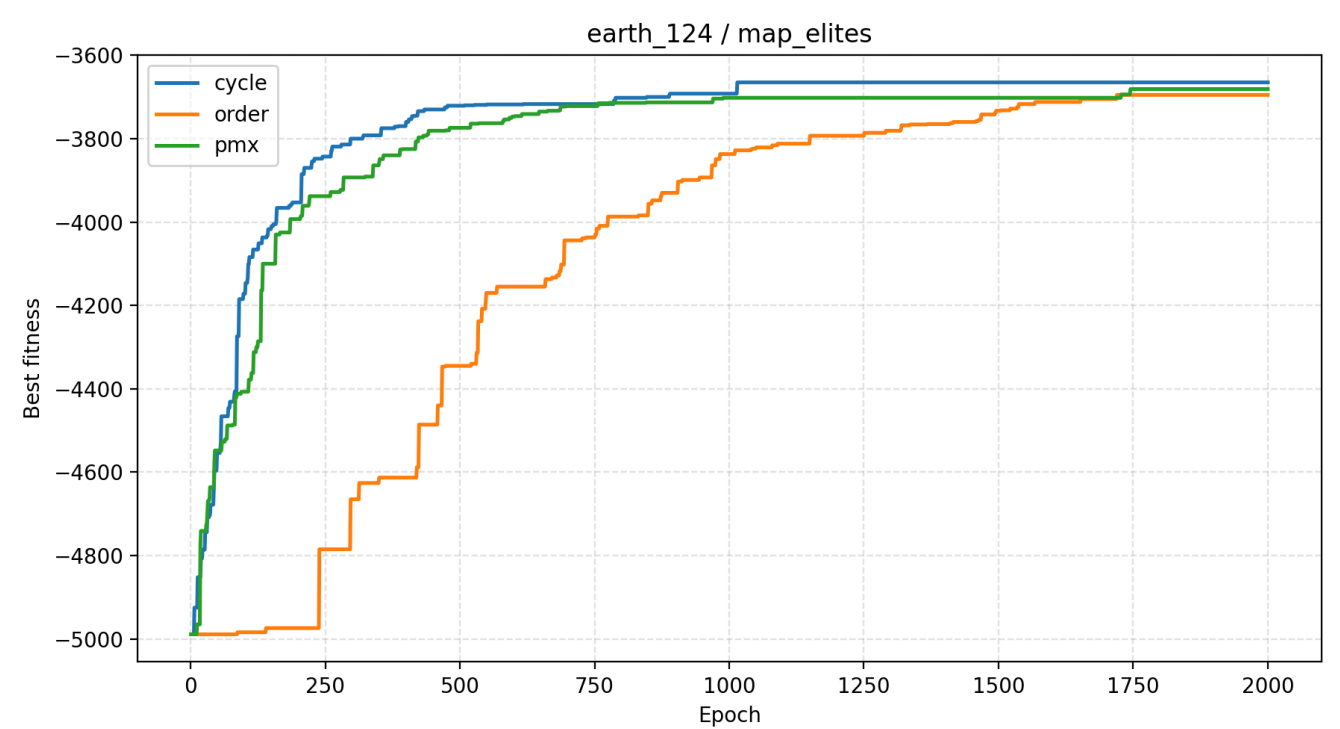
- `algorithm = map_elites`
- `selection_type = tournament`

- `tournament_size = 3`
- `mutation_type = mix`
- `correlation_threshold = 0.68`
- `archive_size = 20`
- `pop_size = 20`
- `init_sampler_type = init_v3`
- 不启用 adaptive mutation、动态 correlation threshold、局部搜索、LNS、threshold accepting 和 diversity bonus。

实验结果保存在：

`results/earth_124/map_elites1/`

对比曲线如下：



各 crossover 算子的最终结果如下：

crossover	total evaluations	archive size	final y_best	排名
cycle	39964	20	-3665	1
pmx	39964	20	-3681	2
order	39964	20	-3695	3

从最终结果看，`cycle crossover` 表现最好，最终达到 `y_best = -3665`，略优于 mutation 对比实验中的最佳 `swap` 结果 `-3666`。`pmx crossover` 最终达到 `-3681`，略弱于之前固定 `insert + pmx` baseline 的 `-3678`，但仍属于较稳定的结果。`order crossover` 最终为 `-3695`，在三者中最弱。

从收敛过程看，三者差异较明显：

epoch	cycle	pmx	order
100	-4172	-4407	-4984
250	-3843	-3938	-4785
500	-3721	-3774	-4345
750	-3717	-3722	-4037
1000	-3692	-3702	-3837
1500	-3665	-3702	-3733
2000	-3665	-3681	-3695

`cycle` 在前期和中期都明显领先，约在 `epoch=1015` 达到 `y_best >= -3678`，并在后期进一步提升到 `-3665`。这说明在 `mutation=mix` 的设置下，保留父代位置循环结构可能更有利于继承有效排序片段，同时保留足够变化。

`pmx` 的表现居中。它能较快进入 `-3800` 到 `-3700` 区间，但后期一度平台化，直到接近实验末期才提升到 `-3681`。这说明 PMX 能稳定继承片段结构，但在该组混合 `mutation` 设置下，后期突破能力略弱于 `cycle`。

`order` 的前期表现最慢，直到 `epoch=913` 才达到 `y_best >= -3900`，明显落后于 `cycle` 和 `pmx`。其原因可能是 `order crossover` 更强调相对顺序继承，而本问题中较好的解不仅依赖相对顺序，也依赖 FAD/LAD 与共生约束修复后的局部位置结构。相对顺序继承过于温和时，配合 `mix mutation` 可能难以快速形成高质量片段。

总体来看，在本次 crossover 对比中，`cycle` 是最优选择，`pmx` 稳定但略弱，`order` 收敛速度和最终值都较差。这也说明在本问题上，保留父代位置结构的 crossover 可能比单纯保留相对顺序更有效。

4. MAP-Elites Baseline 与改进思路

本阶段实验都在 `earth_124 / map_elites` 上进行，保留前面已经实现的锦标赛父代选择：

1. 从 archive 中随机采样 `tournament_size` 个候选父代。
2. 选择其中 fitness 最高者作为父代。
3. 对第二个父代重复该过程，并尽量避免两个父代相同。

该选择方式只使用 archive 中已经保存的 fitness，不会增加 benchmark evaluation 次数。本组 baseline 的配置为：

- `selection_type = tournament`
- `tournament_size = 3`
- `mutation_type = mix`
- `crossover_type = cycle`
- `correlation_threshold = 0.68`
- `archive_size = 20`
- `pop_size = 20`
- `init_sampler_type = init_v3`

baseline 结果保存在：

`results/earth_124/map_elites/baseline/`

其最终结果为：

- total_evaluations = 39964
- local_search_evaluations = 0
- final_y_best = -3749

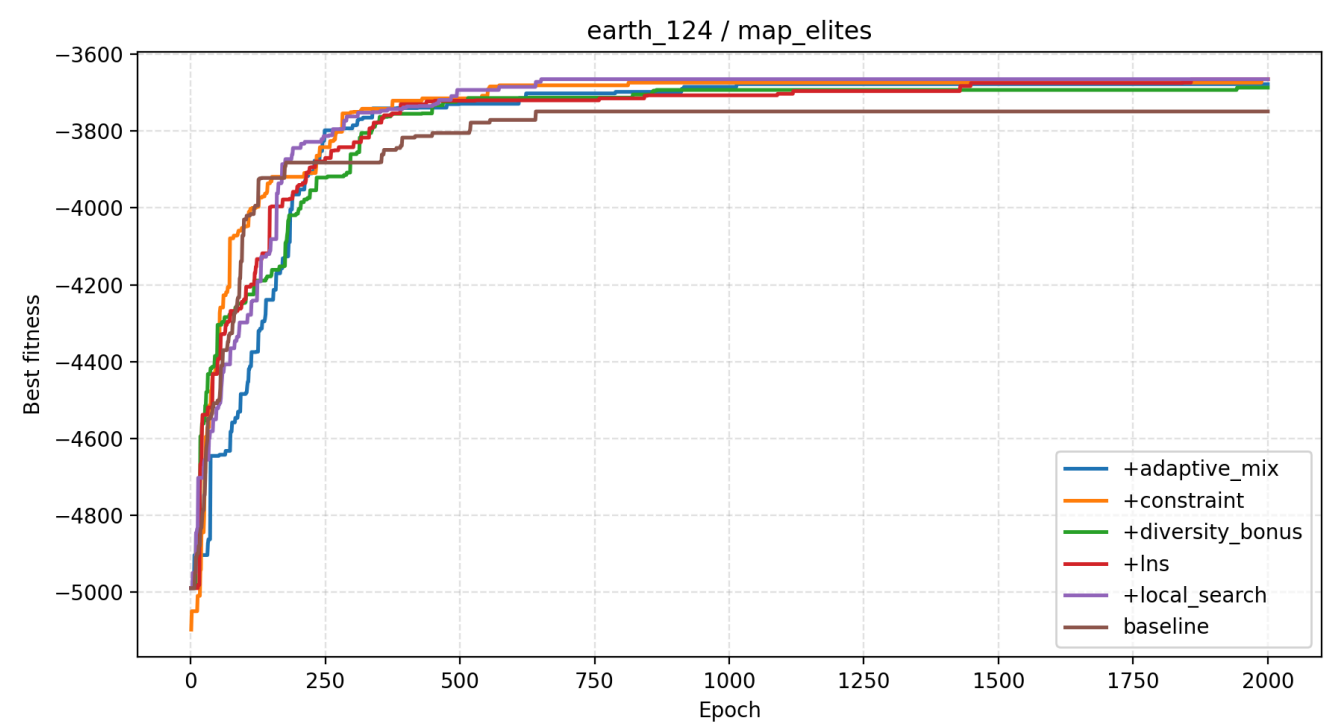
后续实验围绕三个方向改进：第一，使用 coex.dat 和 Fb4L.dat 构造约束引导算子；第二，使用 LNS 提供中后期的大邻域结构扰动；第三，在后期加入少量 local search 做精修。

5. 单项改进实验

单项改进实验结果位于：

results/earth_124/map_elites/

对比曲线如下：



实验结果汇总如下：

方法	核心配置	total evaluations	local search evals	final y_best	首次达到最终值 epoch
Baseline	<code>mix + cycle</code>	39964	0	-3749	641
<code>+constraint</code>	<code>constraint_insert + cycle</code> <code>+ constraint_repair</code>	39964	0	-3665	1989
<code>+adaptive_mix</code>	<code>adaptive_mix + cycle</code>	39964	0	-3678	1014
<code>+local_search</code>	<code>insert + cycle + dynamic threshold</code> <code>+ local_search</code>	40124	160	-3665	651
<code>+lns</code>	<code>insert + cycle + LNS</code>	39964	0	-3665	1858

方法	核心配置	total evaluations	local search evals	final y_best	首次达到 最终值 epoch
+diversity_bonus	insert + cycle + threshold accepting + diversity bonus	39964	0	-3687	1943

5.1 Constraint 引导算子

+constraint 使用 coex.dat 和 Fb4L.dat 构造 event-level precedence constraints。转换方式为：

```
1 coex(a, b):
2     FAD_a < LAD_b
3     FAD_b < LAD_a
4
5 Fb4L(a, b):
6     FAD_a < LAD_b
```

转换后每条约束统一表示为 event_a must appear before event_b。在 earth_124 上，coex.dat 有 348 条共生关系，Fb4L.dat 有 1906 条 FAD/LAD 关系，转换并去重后得到 1906 条 event-level 约束。

我们实现了两个组件：

- constraint_repair：在子代评价前扫描违反的 precedence constraints，并通过插入操作将前驱 event 移到后继 event 前面。
- constraint_insert：mutation 时优先选择当前解中违反的约束进行修复；若当前没有违反约束，则退化为普通 insert mutation。

该方法不修改 benchmark，不额外调用 fitness evaluation。实验中 +constraint 的最终值达到 -3665，与本组最优结果持平，并且 evaluation 数仍为 39964。从曲线看，+constraint 前期收敛稳定，epoch 500 已达到 -3715，后期仍有持续改进，直到 epoch 1989 达到最终值 -3665。这说明约束引导算子不只是前期减少明显冲突，也能在后期继续提供有方向的细粒度调整。

5.2 Adaptive Mix

adaptive_mix 在 swap/insert/reversal/shuffle/shift 之间动态选择 mutation。选择概率由两部分组成：一是阶段性先验，前期提高探索型 mutation 权重，后期提高 insert/swap 权重；二是历史成功率，某种 mutation 产生的子代越常进入 archive，后续权重越高。

该方法最终达到 -3678，明显优于 baseline，但没有达到 +constraint、+lns 和 +local_search 的最终值。原因在于 adaptive mix 学到的是“哪类普通扰动更容易成功”，但普通扰动本身没有直接利用共生和 FAD/LAD 约束结构；当搜索进入平台后，仅靠调整基础 mutation 概率仍然不足以稳定突破。

5.3 Local Search

+local_search 使用 insert + cycle 作为主搜索，并开启动态 correlation_threshold 和局部搜索。局部搜索每隔固定 epoch 触发一次，从当前 batch 中选择高分候选，生成少量 insert/swap 邻居并额外评价。

该方法最终达到 `-3665`，并且首次达到最终值的 epoch 是 651，是单项实验中最快的。其代价是额外进行了 160 次 local search evaluation，总评价数为 `40124`。从曲线看，local search 的优势主要体现在早期和中期，它能快速把当前较好的候选推到附近局部最优；但达到 `-3665` 后基本平台化，说明其主要作用是加速 exploitation，而不是持续提供跨区域探索。

5.4 LNS Destroy-Repair

`+lns` 使用 `insert + cycle`，并开启 LNS destroy-repair。该算子随机选择一小组 taxon，删除它们的 FAD/LAD event，再重新插入回序列，从而一次性重构一个较大的局部邻域。

该方法不额外调用 benchmark，最终同样达到 `-3665`。它的特点是前期不如 local search 快，但中后期仍然能持续改进：epoch 1000 为 `-3707`，epoch 1500 为 `-3675`，直到 epoch 1858 达到 `-3665`。这说明 LNS 的价值主要在中后期，当普通 mutation 已经开始平台化时，大邻域扰动仍可能带来新的结构组合。

5.5 Threshold Accepting 与 Diversity Bonus

`+diversity_bonus` 同时开启 threshold accepting 和 diversity bonus。前者允许前中期接收一部分略差但相似度较高的候选，后者根据 `1 - max_correlation` 给结构差异更大的候选一个逐步衰减的接收加分。

该方法最终达到 `-3687`，优于 baseline，但弱于 constraint、LNS 和 local search。它在 archive 接收层面鼓励多样性，但没有直接改变子代生成方式。因此它能扩大搜索覆盖范围，却不一定能生成更符合问题结构的高质量候选。

6. Phased Best 算法设计

单项实验显示，不同改进项在不同阶段的作用不同：

- `constraint`：前期能将搜索引导到更符合共生和 FAD/LAD 约束的区域，后期也能继续做有方向的小调整。
- `LNS`：中后期仍能通过大邻域重构提供新的结构组合。
- `local_search`：可以快速精修当前高质量候选，但会增加 evaluation，且更偏 exploitation。

因此我们没有把所有机制从头到尾同时开启，而是设计了一个分阶段的 best 算法。实现位置为：

- `main_best.py`
- `scripts/run_earth_124_best1.sh`

算法流程为：

```
1 Phase 1: epoch 1-500
2     mutation_type = constraint_insert
3     constraint_repair_enabled = true
4     lns_enabled = false
5     local_search_enabled = false
6
7 Phase 2: epoch 501-1500
8     mutation_type = insert
9     constraint_repair_enabled = true
10    lns_enabled = true
11    local_search_enabled = false
12
13 Phase 3: epoch 1501-2000
14    mutation_type = insert
15    constraint_repair_enabled = true
```

```
16 | lns_enabled = true
17 | local_search_enabled = true
```

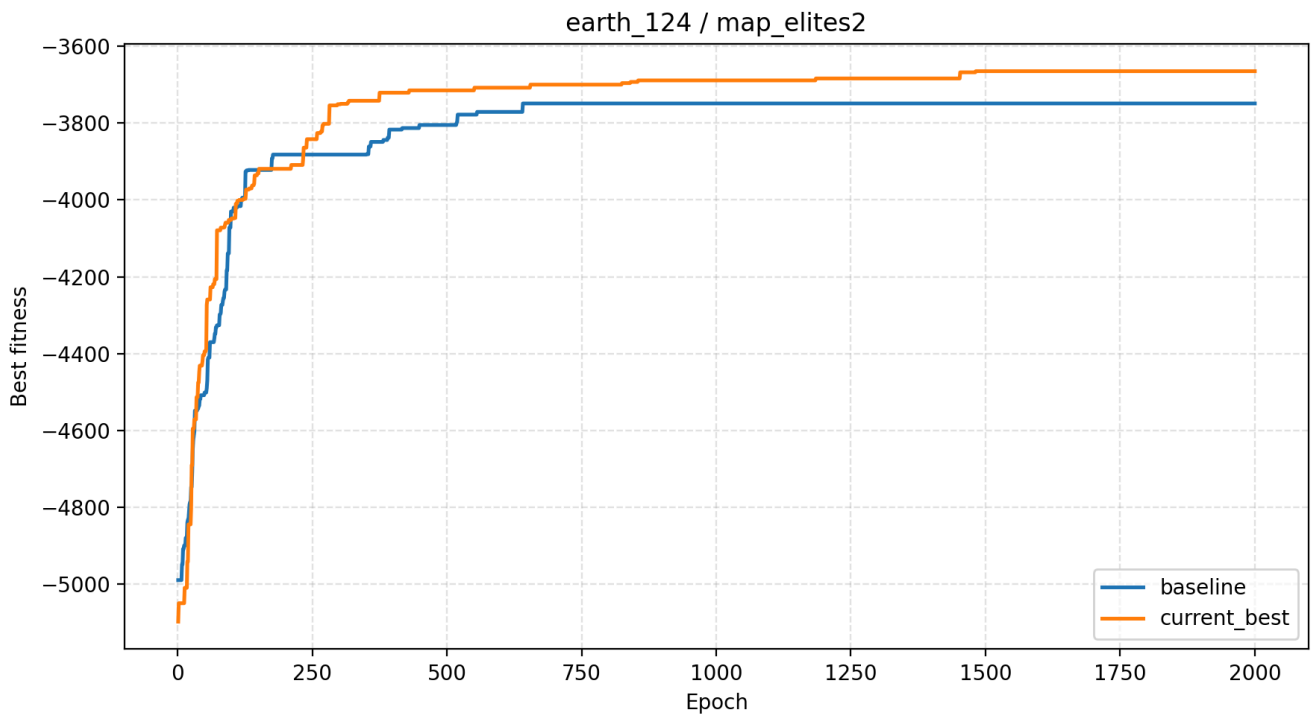
这个设计的核心是让每个阶段只承担一个主要目标：前期先提高可行结构质量，中期用大邻域扰动延缓平台化，后期再用少量 local search 做精修。constraint_repair 全程保留，因为它不增加 evaluation，且能持续把候选拉回更符合问题约束的区域。LNS 从中期开始启用，避免前期 archive 尚未稳定时过多破坏结构。local search 只在后期启用，避免过早把种群推入相似局部最优。

7. Best 算法实验结果

best 算法结果保存在：

results/earth_124/map_elites2/current_best/

为了让曲线更清楚，我们在 map_elites2 目录下单独绘制 baseline 与 current_best 的对比图。两个目录中的 baseline 配置相同。



结果汇总如下：

方法	total evaluations	local search evals	final archive size	final y_best	首次达到最终值 epoch
Baseline	39964	0	3	-3749	641
Current best	40004	40	20	-3665	1482

current best 最终达到 -3665，与单项实验中的 +constraint、+lns 和 +local_search 并列为当前最好最终值。相比单独 local search，它只额外使用了 40 次 local search evaluation，而不是 160 次；相比单独 +constraint，它更早达到最终值，epoch 1482 达到 -3665，而 +constraint 是 epoch 1989；相比单独 +lns，它也更早达到最终值，+lns 是 epoch 1858。

从阶段曲线看，current best 的行为符合设计预期：

- epoch 1-500：使用 constraint 阶段，epoch 500 达到 `-3715`，为后续搜索提供较好的约束结构基础。
- epoch 501-1500：进入 LNS 阶段，继续从 `-3715` 改进到 `-3665`，说明中期大邻域扰动成功提供了新的结构组合。
- epoch 1501-2000：进入 local search 阶段后保持最优值，说明后期精修没有破坏已有 elite，同时用较少额外 evaluation 维持搜索稳定。

因此，current best 的优势并不在于把所有改进简单叠加，而在于按演化阶段分配搜索压力：前期约束引导，中期结构跳跃，后期局部精修。这个策略与本问题的排列约束特征相匹配，也解释了为什么全程叠加多个机制未必更好，而分阶段使用可以更稳定地达到当前最好结果。

8. 当前结论与不足

本阶段 MAP-Elites 实验表明，单纯调节基础 mutation 概率的收益有限；更有效的方向是利用问题本身的结构信息，尤其是 `coex.dat` 与 `Fb4L.dat` 中隐含的 event precedence constraints。 `constraint_insert + constraint_repair` 在不增加 evaluation 的情况下达到 `-3665`，说明约束引导确实能提高子代质量。

LNS 和 local search 分别对应 exploration 与 exploitation：LNS 更适合中后期突破平台，local search 更适合快速精修。current best 将 constraint、LNS、local search 分阶段使用，最终同样达到 `-3665`，并且只额外增加 40 次 local search evaluation，是目前最合理的综合方案。

当前结果仍基于单个 seed。后续如果时间允许，应对 baseline、`+constraint`、`+lns`、`+local_search` 和 current best 做多 seed 重复实验，比较均值、标准差和最优值，以确认该分阶段策略的稳定性。

9. 任务二：RLEA 强化学习演化算子

任务二关注基于强化学习的演化算子。与任务一中手工设计 `swap/insert/reversal/shuffle/shift` 等变异算子不同，RLEA 的核心思想是：让一个强化学习策略在两个父代解的基础上，逐步构造一个子代排列，从而用学习到的策略替代部分传统 `crossover + mutation` 操作。

9.1 RLEA 的基本原理

传统 EA 的子代生成流程通常是：

```
1 选择父代 x1, x2
2 执行 crossover
3 执行 mutation
4 得到 offspring
```

RLEA 则在部分 epoch 中改为：

```
1 选择父代 x1, x2
2 将父代及其 fitness 输入 RL 环境
3 RL policy 逐步选择 action
4 环境根据 action 构造 offspring
5 返回 RL 生成的 offspring
```

在当前项目中，RLEA 使用 `algorithms/_ea.py` 中的 `EA` 类实现。该类同时支持传统演化算子和 RL 算子。当 `use_rl=False` 时，仍然执行普通 `crossover + mutation`；当 `use_rl=True` 时，调用 `_neural_crossover_and_mutation()`，由训练好的 Q 网络生成子代。

9.2 代码实现路径

RLEA 的推理路径如下：

```
1 | scripts/run_earth_124_rlea.sh
2 | -> main.py
3 | -> algorithms/_ea.py
4 | -> EA.ask()
5 | -> EA._neural_crossover_and_mutation()
6 | -> Environments/CONOPLib_env.py
7 | -> trained Q network
```

其中 `main.py` 控制何时启用 RL。我们为其增加了 `rl_freq` 参数：

```
1 | use_rl = is_rl_alg and rl_freq > 0 and epoch % rl_freq == 0
```

因此，RLEA 并不是必须每一轮都使用 RL。`rl_freq=500` 表示每 500 个 epoch 使用一次 RL 算子，`rl_freq=100` 表示每 100 个 epoch 使用一次，`rl_freq=1` 则表示每一轮都使用 RL。其余 epoch 仍然使用传统 EA 的 `crossover + mutation`。这样做的好处是可以把 RL 作为一种周期性强化算子，而不是完全替代传统演化搜索。

9.3 父代选择与生存选择

为了与任务一中的改进保持一致，我们为 `_ea.py` 增加了锦标赛父代选择。RLEA 当前使用两层选择机制：

- 父代选择：`parent_selection_type = tournament`
- 生存选择：`selection_type = rank_based_prioritized`

父代选择用于决定哪两个个体进入 `crossover` 或 RL 算子。锦标赛选择只使用当前 `population` 中已有的 `fitness`，不增加 `benchmark evaluation` 次数。

生存选择用于从 `parent population` 和 `offspring` 中选择下一代。`rank_based_prioritized` 根据 `fitness` 排名分配采样概率，高 `fitness` 个体更容易保留，但低排名个体仍有一定概率进入下一代，从而保留部分多样性。

9.4 RL 训练过程

RLEA 推理前需要先训练一个 `policy checkpoint`。训练脚本为：

```
1 | train.py
```

训练前还需要准备 RL 环境使用的父代池：

```
1 | data/earth_124/gen_points_v3/
```

我们新增了脚本：

```
1 | scripts/generate_rl_parents.py
```

用于生成训练环境所需的父代解。生成命令示例：

```
1 | python scripts/generate_rl_parents.py \  
2 |   --dims 124 \  
3 |   --num-parents 50 \  
4 |   --seed 2022 \  
5 |   --sampler init_v3
```

训练命令示例：

```
1 | python train.py \  
2 |   --task_name EarthBenchEnv_124 \  
3 |   --epoch 2000 \  
4 |   --train_envs 100 \  
5 |   --test_envs 100 \  
6 |   --save_path multi_envs_models/dim124_epoch2000.pth \  
7 |   --device cuda:0
```

训练过程中，Q 网络在 GPU 上更新参数，环境交互和 CONOP 相关计算主要在 CPU 上进行。训练完成后，会保存 checkpoint，例如：

```
1 | multi_envs_models/dim124_epoch2000.pth
```

该 checkpoint 中包含：

- `model_state_dict`：Q 网络参数。
- `policy_state_dict`：Tianshou policy 参数。
- `model_config`：网络结构配置，如 `dims`、`action_dim`、`d_model`、`nhead`、`num_layers`。
- `train_config`：训练超参数。

9.5 RLEA 推理运行过程

我们新增了专门的运行脚本：

```
1 | scripts/run_earth_124_rlea.sh
```

快速测试命令为：

```
1 | bash ./scripts/run_earth_124_rlea.sh \  
2 |   --policy-path multi_envs_models/dim124_test.pth \  
3 |   --epochs 20 \  
4 |   --device cuda:0
```

正式运行命令为：

```
1 bash ./scripts/run_earth_124_rlea.sh \  
2 --policy-path multi_envs_models/dim124_epoch2000.pth \  
3 --epochs 2000 \  
4 --device cuda:0 \  
5 --rl-freq 100
```

该脚本默认配置如下：

- `algorithm = RLEA`
- `mutation_type = insert`
- `crossover_type = pmx`
- `parent_selection_type = tournament`
- `tournament_size = 3`
- `selection_type = rank_based_prioritized`
- `init_sampler_type = init_v3`
- `pop_size = 20`

推理过程中，普通 benchmark evaluation 仍由 `main.py` 中的 Ray worker 调用 `EarthBenchmark` 完成，主要运行在 CPU 上；RL policy 的神经网络前向推理运行在 `cuda:0` 上。

运行结果会保存到：

```
1 | results/earth_124/RLEA/<timestamp>/
```

其中包括：

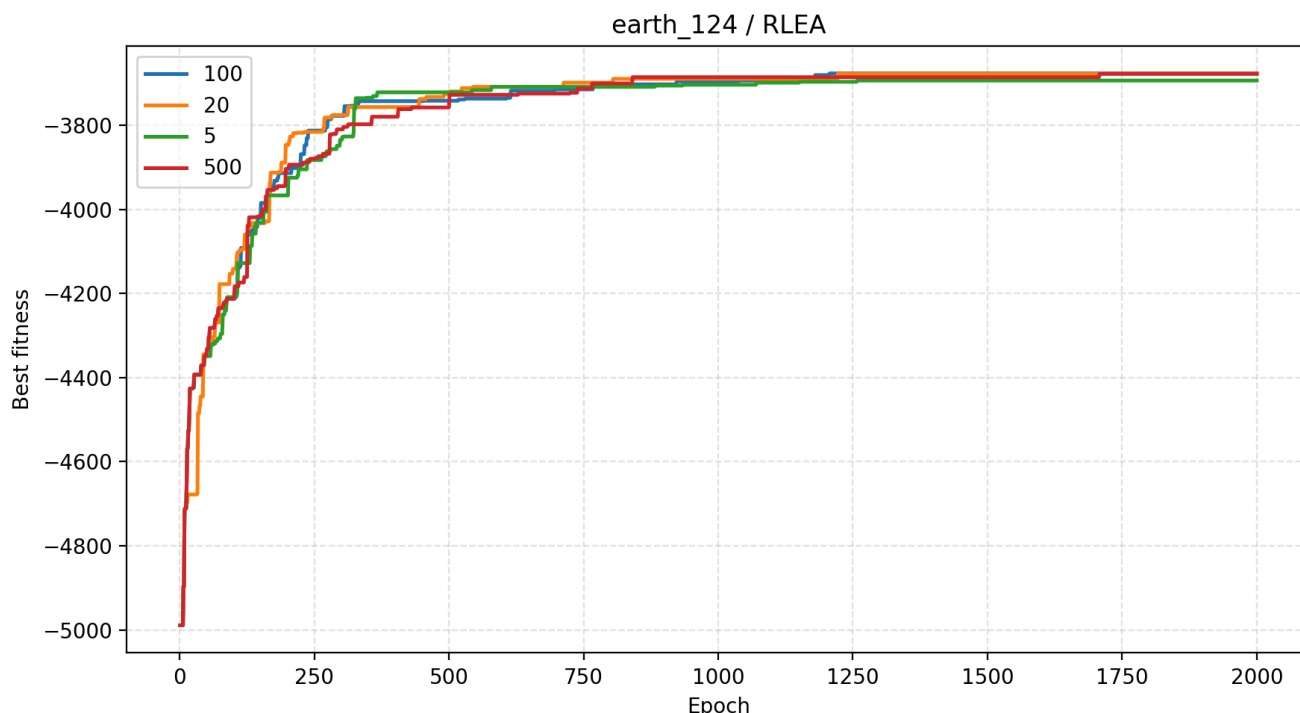
- `metrics.csv`
- `config.json`
- `final_summary.json`
- `y_best_curve.png`

9.6 不同 RL 使用频率的实验比较

为了比较 RL 算子在 RLEA 中的使用频率，我们对 `rl_freq` 做了消融实验。`rl_freq` 表示每隔多少个 epoch 启用一次 RL operator。数值越大，RL 介入越少；数值越小，RL 介入越频繁。

本组实验均使用 `earth_124`、`RLEA`、`pop_size=20`、`pmx` crossover、`insert` mutation、锦标赛父代选择和 `rank_based_prioritized` 生存选择。每组均运行 2000 epoch，总 benchmark evaluation 数均为 40000，因此不同曲线之间可以直接比较最终 `y_best` 和收敛速度。

对比曲线如下：



当前结果汇总如下：

rl_freq	policy checkpoint	最终 y_best	首次达到最终值的 epoch	中期表现
500	dim124_epoch2000.pth	-3678	1708	epoch 500 时为 -3758, epoch 1000 时为 -3686
100	dim124_epoch2000.pth	-3677	1207	epoch 500 时为 -3742, epoch 1000 时为 -3696
20	dim124_epoch2000.pth	-3678	1222	epoch 500 时为 -3726, epoch 1000 时为 -3690
5	dim124_epoch2000.pth	-3694	1258	epoch 500 时为 -3722, epoch 1000 时为 -3704

从曲线看，四组实验前期收敛趋势非常接近，都能较快从约 -5000 提升到 -37xx 区间。主要差异出现在中后期平台阶段：更频繁地使用 RL 并没有带来持续突破，反而可能让搜索更早进入相似结构附近。

rl_freq=500 在 2000 epoch 内只启用约 4 次 RL operator，RL 对整体演化的影响较弱，因此最终表现接近普通 EA，停在 -3678。它的后期仍有一次较晚提升，在 epoch 1708 达到最终值，但总体收敛速度不算最好。

rl_freq=100 在 2000 epoch 内约启用 20 次 RL operator，最终达到 -3677，是本组实验中的最好结果。它在 epoch 1207 就达到最终值，相比 500 更早进入最终平台，并且最终值略高。这说明适度使用 RL 可以给传统 pmx + insert 的演化流程提供额外重组能力，但这个提升幅度仍然很小。

rl_freq=20 的最终结果为 -3678，与 500 基本持平。它在中期一度更快接近较好区域，例如 epoch 500 时达到 -3726，优于 500 和 100；但后期没有继续突破，最终仍停在同一平台附近。这说明提高 RL 使用频率主要改善了部分中期收敛速度，并没有稳定改善最终最优值。

`r1_freq=5` 的最终结果为 `-3694`，明显弱于其他三组。该配置下 RL operator 介入过于频繁，传统 EA 中 crossover/mutation 的随机探索空间被压缩。如果 RL policy 本身还没有学到足够强的全局重组策略，过高频率地使用它会使种群更容易沿着同一种策略偏好收敛，导致多样性下降，最终结果反而变差。

综合当前单 seed 实验，`r1_freq=100` 是相对最好的配置。它在 RL 介入强度和传统 EA 探索之间取得了较好的平衡：比 `500` 有更多学习型重组机会，又不像 `5` 那样让 RL policy 过度主导演化过程。不过，当前 RLEA 的提升幅度只有 `1` 个 fitness 单位，说明 RL operator 尚未稳定突破传统 EA 的 plateau。后续如果继续优化 RLEA，重点不应只放在调整 `r1_freq`，还需要改进 RL 训练质量、reward 设计、父代分布构造，以及让 policy 学习更有跳出局部 basin 能力的动作。